

Environments and Deployment Guide

Configuration, deployment, validation and rollback.



This document explains its subject first in plain language, then in engineering detail. It is grounded in the Craves repository snapshot and deliberately separates current implementation, legacy/reference code, milestone foundations and repository gaps so readers do not mistake aspiration for proof.

Version 1.0 | Evidence date: 14 August 2026 | Repository: [rmorampudi09-arch/Craves-Build-platform](#) | Snapshot commit: [3225f7fa531a6b07185fb5e034f7930f8bd8b571](#)

pipeline evidence method - plain-language purpose

Plain-English explanation

Craves includes a governed semantic meal concierge for assisted discovery rather than an authority that can bypass commerce rules.

How it works technically

Recent main history and apps/customer-web-next/docs/signalr-ai-concierge.md describe SignalR semantic concierge work. Recommendations still rely on governed product data and ordinary catalog, quote, authorization and order paths for transactions.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The ai boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/docs/signalr-ai-concierge.md; commit 71e795b. Snapshot: main @ 3225f7fa531a (14 August 2026).

pipeline evidence method - technical architecture

Plain-English explanation

The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls.

How it works technically

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The devops boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Snapshot: main @ 3225f7fa531a (14 August 2026).

pipeline evidence method - end-to-end flow

Plain-English explanation

The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls.

How it works technically

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The devops boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Snapshot: main @ 3225f7fa531a (14 August 2026).

pipeline evidence method - errors and edge cases

Plain-English explanation

The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls.

How it works technically

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The devops boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Snapshot: main @ 3225f7fa531a (14 August 2026).

pipeline evidence method - deployment, security and operational validation

Plain-English explanation

The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls.

How it works technically

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The devops boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Snapshot: main @ 3225f7fa531a (14 August 2026).

backend CI - plain-language purpose

Plain-English explanation

Craves includes a governed semantic meal concierge for assisted discovery rather than an authority that can bypass commerce rules.

How it works technically

Recent main history and [apps/customer-web-next/docs/signalr-ai-concierge.md](#) describe SignalR semantic concierge work. Recommendations still rely on governed product data and ordinary catalog, quote, authorization and order paths for transactions.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The ai boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: [apps/customer-web-next/docs/signalr-ai-concierge.md](#); commit 71e795b. Snapshot: main @ 3225f7fa531a (14 August 2026).

backend CI - technical architecture

Plain-English explanation

The active backend target is seven Spring Boot services: auth, user-chef, catalog, order, subscription, integration and notification.

How it works technically

Services own focused business domains, validate JWT role/ownership, expose health endpoints, evolve schema through Flyway where documented, and isolate provider-specific behavior behind integration/notification boundaries.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The backend boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: services/auth-service/README.md; services/user-chef-service/README.md; services/catalog-service/README.md; services/order-service/README.md; services/subscription-service/README.md; services/integration-service/README.md; services/notification-service/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

backend CI - end-to-end flow

Plain-English explanation

The active backend target is seven Spring Boot services: auth, user-chef, catalog, order, subscription, integration and notification.

How it works technically

Services own focused business domains, validate JWT role/ownership, expose health endpoints, evolve schema through Flyway where documented, and isolate provider-specific behavior behind integration/notification boundaries.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The backend boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: services/auth-service/README.md; services/user-chef-service/README.md; services/catalog-service/README.md; services/order-service/README.md; services/subscription-service/README.md; services/integration-service/README.md; services/notification-service/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

backend CI - errors and edge cases

Plain-English explanation

A useful failure model separates user input/state conflicts, identity/permission failures, dependency outages, data faults and deployment/configuration incidents.

How it works technically

order-service documents Problem Details fields and codes including ORDER_NOT_FOUND, ORDER_IDEMPOTENCY_CONFLICT, INVALID_STATUS, CANCELLATION_NOT_ALLOWED, REFUND_FAILED, UNAUTHORIZED, FORBIDDEN, BACKEND_CHANGE_DISABLED and BACKEND_UNAVAILABLE. Diagnostics should start from request/correlation evidence.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The errors boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: services/order-service/README.md; services/notification-service/README.md; apps/customer-web-next/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

backend CI - deployment, security and operational validation

Plain-English explanation

The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls.

How it works technically

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The devops boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Snapshot: main @ 3225f7fa531a (14 August 2026).

frontend CI - plain-language purpose

Plain-English explanation

Craves includes a governed semantic meal concierge for assisted discovery rather than an authority that can bypass commerce rules.

How it works technically

Recent main history and [apps/customer-web-next/docs/signalr-ai-concierge.md](#) describe SignalR semantic concierge work. Recommendations still rely on governed product data and ordinary catalog, quote, authorization and order paths for transactions.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The ai boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: [apps/customer-web-next/docs/signalr-ai-concierge.md](#); commit 71e795b. Snapshot: main @ 3225f7fa531a (14 August 2026).

frontend CI - technical architecture

Plain-English explanation

The primary customer-facing web implementation is apps/customer-web-next, a Next.js 14 App Router application containing customer, chef and newer admin/support modules.

How it works technically

The browser uses same-origin BFF handlers, secure HTTP-only token cookies and APIM at /api/v1. Firebase phone OTP is used for the current web sign-in flow, and backend responses remain authoritative for price, availability and order state.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The frontend boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/app/; apps/customer-web-next/src/components/; apps/customer-web-next/src/features/. Snapshot: main @ 3225f7fa531a (14 August 2026).

frontend CI - end-to-end flow

Plain-English explanation

The primary customer-facing web implementation is apps/customer-web-next, a Next.js 14 App Router application containing customer, chef and newer admin/support modules.

How it works technically

The browser uses same-origin BFF handlers, secure HTTP-only token cookies and APIM at /api/v1. Firebase phone OTP is used for the current web sign-in flow, and backend responses remain authoritative for price, availability and order state.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The frontend boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/app/; apps/customer-web-next/src/components/; apps/customer-web-next/src/features/. Snapshot: main @ 3225f7fa531a (14 August 2026).

frontend CI - errors and edge cases

Plain-English explanation

A useful failure model separates user input/state conflicts, identity/permission failures, dependency outages, data faults and deployment/configuration incidents.

How it works technically

order-service documents Problem Details fields and codes including ORDER_NOT_FOUND, ORDER_IDEMPOTENCY_CONFLICT, INVALID_STATUS, CANCELLATION_NOT_ALLOWED, REFUND_FAILED, UNAUTHORIZED, FORBIDDEN, BACKEND_CHANGE_DISABLED and BACKEND_UNAVAILABLE. Diagnostics should start from request/correlation evidence.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The errors boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: services/order-service/README.md; services/notification-service/README.md; apps/customer-web-next/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

frontend CI - deployment, security and operational validation

Plain-English explanation

The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls.

How it works technically

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The devops boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Snapshot: main @ 3225f7fa531a (14 August 2026).

admin CI - plain-language purpose

Plain-English explanation

Craves includes a governed semantic meal concierge for assisted discovery rather than an authority that can bypass commerce rules.

How it works technically

Recent main history and [apps/customer-web-next/docs/signalr-ai-concierge.md](#) describe SignalR semantic concierge work. Recommendations still rely on governed product data and ordinary catalog, quote, authorization and order paths for transactions.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The ai boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: [apps/customer-web-next/docs/signalr-ai-concierge.md](#); commit 71e795b. Snapshot: main @ 3225f7fa531a (14 August 2026).

admin CI - technical architecture

Plain-English explanation

Administrative tooling lets authorized staff review marketplace state unavailable to ordinary customers and chefs.

How it works technically

The repository contains legacy apps/admin plus newer customer-web-next admin/support/chef-review modules. APIM guidance places admin operations in a subscription-protected product with additional role checks.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The admin boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/admin/README.md; apps/customer-web-next/src/features/admin/; apps/customer-web-next/src/features/chef-review/; infra/apim/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

admin CI - end-to-end flow

Plain-English explanation

Administrative tooling lets authorized staff review marketplace state unavailable to ordinary customers and chefs.

How it works technically

The repository contains legacy apps/admin plus newer customer-web-next admin/support/chef-review modules. APIM guidance places admin operations in a subscription-protected product with additional role checks.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The admin boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/admin/README.md; apps/customer-web-next/src/features/admin/; apps/customer-web-next/src/features/chef-review/; infra/apim/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

admin CI - errors and edge cases

Plain-English explanation

Administrative tooling lets authorized staff review marketplace state unavailable to ordinary customers and chefs.

How it works technically

The repository contains legacy apps/admin plus newer customer-web-next admin/support/chef-review modules. APIM guidance places admin operations in a subscription-protected product with additional role checks.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The admin boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/admin/README.md; apps/customer-web-next/src/features/admin/; apps/customer-web-next/src/features/chef-review/; infra/apim/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

admin CI - deployment, security and operational validation

Plain-English explanation

Administrative tooling lets authorized staff review marketplace state unavailable to ordinary customers and chefs.

How it works technically

The repository contains legacy apps/admin plus newer customer-web-next admin/support/chef-review modules. APIM guidance places admin operations in a subscription-protected product with additional role checks.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The admin boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/admin/README.md; apps/customer-web-next/src/features/admin/; apps/customer-web-next/src/features/chef-review/; infra/apim/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

mobile CI - plain-language purpose

Plain-English explanation

Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot.

How it works technically

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The mobile boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Milestone foundation. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

mobile CI - technical architecture

Plain-English explanation

Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot.

How it works technically

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The mobile boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Milestone foundation. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

mobile CI - end-to-end flow

Plain-English explanation

Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot.

How it works technically

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The mobile boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Milestone foundation. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

mobile CI - errors and edge cases

Plain-English explanation

Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot.

How it works technically

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The mobile boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Milestone foundation. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

mobile CI - deployment, security and operational validation

Plain-English explanation

Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot.

How it works technically

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The mobile boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Milestone foundation. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

APIM CI - plain-language purpose

Plain-English explanation

Azure API Management is the intended policy and routing front door between clients/BFFs and backend services.

How it works technically

The APIM README describes craves-customer without APIM subscription requirement and craves-admin with subscription required, JWT validation against Entra and operation-level role checks from named values. Its cited OpenAPI path was not resolvable on the reviewed main snapshot and is documented as a gap.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The apim boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: infra/apim/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

APIM CI - technical architecture

Plain-English explanation

Azure API Management is the intended policy and routing front door between clients/BFFs and backend services.

How it works technically

The APIM README describes craves-customer without APIM subscription requirement and craves-admin with subscription required, JWT validation against Entra and operation-level role checks from named values. Its cited OpenAPI path was not resolvable on the reviewed main snapshot and is documented as a gap.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The apim boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: infra/apim/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

APIM CI - end-to-end flow

Plain-English explanation

Azure API Management is the intended policy and routing front door between clients/BFFs and backend services.

How it works technically

The APIM README describes craves-customer without APIM subscription requirement and craves-admin with subscription required, JWT validation against Entra and operation-level role checks from named values. Its cited OpenAPI path was not resolvable on the reviewed main snapshot and is documented as a gap.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The apim boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: infra/apim/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

APIM CI - errors and edge cases

Plain-English explanation

Azure API Management is the intended policy and routing front door between clients/BFFs and backend services.

How it works technically

The APIM README describes craves-customer without APIM subscription requirement and craves-admin with subscription required, JWT validation against Entra and operation-level role checks from named values. Its cited OpenAPI path was not resolvable on the reviewed main snapshot and is documented as a gap.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The apim boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: infra/apim/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

APIM CI - deployment, security and operational validation

Plain-English explanation

Azure API Management is the intended policy and routing front door between clients/BFFs and backend services.

How it works technically

The APIM README describes craves-customer without APIM subscription requirement and craves-admin with subscription required, JWT validation against Entra and operation-level role checks from named values. Its cited OpenAPI path was not resolvable on the reviewed main snapshot and is documented as a gap.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The apim boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: infra/apim/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

data migration CI - plain-language purpose

Plain-English explanation

Craves includes a governed semantic meal concierge for assisted discovery rather than an authority that can bypass commerce rules.

How it works technically

Recent main history and apps/customer-web-next/docs/signalr-ai-concierge.md describe SignalR semantic concierge work. Recommendations still rely on governed product data and ordinary catalog, quote, authorization and order paths for transactions.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The ai boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/docs/signalr-ai-concierge.md; commit 71e795b. Snapshot: main @ 3225f7fa531a (14 August 2026).

data migration CI - technical architecture

Plain-English explanation

The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls.

How it works technically

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The devops boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Snapshot: main @ 3225f7fa531a (14 August 2026).

data migration CI - end-to-end flow

Plain-English explanation

The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls.

How it works technically

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The devops boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Snapshot: main @ 3225f7fa531a (14 August 2026).

data migration CI - errors and edge cases

Plain-English explanation

A useful failure model separates user input/state conflicts, identity/permission failures, dependency outages, data faults and deployment/configuration incidents.

How it works technically

order-service documents Problem Details fields and codes including ORDER_NOT_FOUND, ORDER_IDEMPOTENCY_CONFLICT, INVALID_STATUS, CANCELLATION_NOT_ALLOWED, REFUND_FAILED, UNAUTHORIZED, FORBIDDEN, BACKEND_CHANGE_DISABLED and BACKEND_UNAVAILABLE. Diagnostics should start from request/correlation evidence.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The errors boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: services/order-service/README.md; services/notification-service/README.md; apps/customer-web-next/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

data migration CI - deployment, security and operational validation

Plain-English explanation

The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls.

How it works technically

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The devops boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Snapshot: main @ 3225f7fa531a (14 August 2026).

privacy/security CI - plain-language purpose

Plain-English explanation

Privacy capability includes consent preferences plus recent customer data export/deletion implementation on main.

How it works technically

Recent commits record centralized consent enforcement, a consent-center UI/backend wiring, customer export and deletion workflows. Exact legal retention periods are not invented where source is silent.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The privacy boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: commit 188385e; commit 93ebfa4; commit 5dd7971; apps/customer-web-next/. Snapshot: main @ 3225f7fa531a (14 August 2026).

privacy/security CI - technical architecture

Plain-English explanation

Privacy capability includes consent preferences plus recent customer data export/deletion implementation on main.

How it works technically

Recent commits record centralized consent enforcement, a consent-center UI/backend wiring, customer export and deletion workflows. Exact legal retention periods are not invented where source is silent.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The privacy boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: commit 188385e; commit 93ebfa4; commit 5dd7971; apps/customer-web-next/. Snapshot: main @ 3225f7fa531a (14 August 2026).

privacy/security CI - end-to-end flow

Plain-English explanation

Privacy capability includes consent preferences plus recent customer data export/deletion implementation on main.

How it works technically

Recent commits record centralized consent enforcement, a consent-center UI/backend wiring, customer export and deletion workflows. Exact legal retention periods are not invented where source is silent.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The privacy boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: commit 188385e; commit 93ebfa4; commit 5dd7971; apps/customer-web-next/. Snapshot: main @ 3225f7fa531a (14 August 2026).

privacy/security CI - errors and edge cases

Plain-English explanation

Privacy capability includes consent preferences plus recent customer data export/deletion implementation on main.

How it works technically

Recent commits record centralized consent enforcement, a consent-center UI/backend wiring, customer export and deletion workflows. Exact legal retention periods are not invented where source is silent.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The privacy boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: commit 188385e; commit 93ebfa4; commit 5dd7971; apps/customer-web-next/. Snapshot: main @ 3225f7fa531a (14 August 2026).

privacy/security CI - deployment, security and operational validation

Plain-English explanation

Privacy capability includes consent preferences plus recent customer data export/deletion implementation on main.

How it works technically

Recent commits record centralized consent enforcement, a consent-center UI/backend wiring, customer export and deletion workflows. Exact legal retention periods are not invented where source is silent.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The privacy boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: commit 188385e; commit 93ebfa4; commit 5dd7971; apps/customer-web-next/. Snapshot: main @ 3225f7fa531a (14 August 2026).

smoke/e2e - plain-language purpose

Plain-English explanation

Craves includes a governed semantic meal concierge for assisted discovery rather than an authority that can bypass commerce rules.

How it works technically

Recent main history and [apps/customer-web-next/docs/signalr-ai-concierge.md](#) describe SignalR semantic concierge work. Recommendations still rely on governed product data and ordinary catalog, quote, authorization and order paths for transactions.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The ai boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: [apps/customer-web-next/docs/signalr-ai-concierge.md](#); commit 71e795b. Snapshot: main @ 3225f7fa531a (14 August 2026).

smoke/e2e - technical architecture

Plain-English explanation

The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls.

How it works technically

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The devops boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Snapshot: main @ 3225f7fa531a (14 August 2026).

smoke/e2e - end-to-end flow

Plain-English explanation

The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls.

How it works technically

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The devops boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Snapshot: main @ 3225f7fa531a (14 August 2026).

smoke/e2e - errors and edge cases

Plain-English explanation

A useful failure model separates user input/state conflicts, identity/permission failures, dependency outages, data faults and deployment/configuration incidents.

How it works technically

order-service documents Problem Details fields and codes including ORDER_NOT_FOUND, ORDER_IDEMPOTENCY_CONFLICT, INVALID_STATUS, CANCELLATION_NOT_ALLOWED, REFUND_FAILED, UNAUTHORIZED, FORBIDDEN, BACKEND_CHANGE_DISABLED and BACKEND_UNAVAILABLE. Diagnostics should start from request/correlation evidence.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The errors boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: services/order-service/README.md; services/notification-service/README.md; apps/customer-web-next/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

smoke/e2e - deployment, security and operational validation

Plain-English explanation

The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls.

How it works technically

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The devops boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Snapshot: main @ 3225f7fa531a (14 August 2026).

release gates - plain-language purpose

Plain-English explanation

Craves includes a governed semantic meal concierge for assisted discovery rather than an authority that can bypass commerce rules.

How it works technically

Recent main history and [apps/customer-web-next/docs/signalr-ai-concierge.md](#) describe SignalR semantic concierge work. Recommendations still rely on governed product data and ordinary catalog, quote, authorization and order paths for transactions.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The ai boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: [apps/customer-web-next/docs/signalr-ai-concierge.md](#); commit 71e795b. Snapshot: main @ 3225f7fa531a (14 August 2026).

release gates - technical architecture

Plain-English explanation

The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls.

How it works technically

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The devops boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Snapshot: main @ 3225f7fa531a (14 August 2026).

release gates - end-to-end flow

Plain-English explanation

The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls.

How it works technically

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The devops boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Snapshot: main @ 3225f7fa531a (14 August 2026).

release gates - errors and edge cases

Plain-English explanation

A useful failure model separates user input/state conflicts, identity/permission failures, dependency outages, data faults and deployment/configuration incidents.

How it works technically

order-service documents Problem Details fields and codes including ORDER_NOT_FOUND, ORDER_IDEMPOTENCY_CONFLICT, INVALID_STATUS, CANCELLATION_NOT_ALLOWED, REFUND_FAILED, UNAUTHORIZED, FORBIDDEN, BACKEND_CHANGE_DISABLED and BACKEND_UNAVAILABLE. Diagnostics should start from request/correlation evidence.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The errors boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: services/order-service/README.md; services/notification-service/README.md; apps/customer-web-next/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

release gates - deployment, security and operational validation

Plain-English explanation

The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls.

How it works technically

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The devops boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Snapshot: main @ 3225f7fa531a (14 August 2026).

pipeline path gaps - plain-language purpose

Plain-English explanation

Craves includes a governed semantic meal concierge for assisted discovery rather than an authority that can bypass commerce rules.

How it works technically

Recent main history and apps/customer-web-next/docs/signalr-ai-concierge.md describe SignalR semantic concierge work. Recommendations still rely on governed product data and ordinary catalog, quote, authorization and order paths for transactions.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The ai boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Repository gap. Evidence: apps/customer-web-next/docs/signalr-ai-concierge.md; commit 71e795b. Snapshot: main @ 3225f7fa531a (14 August 2026).

pipeline path gaps - technical architecture

Plain-English explanation

The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls.

How it works technically

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The devops boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Repository gap. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Snapshot: main @ 3225f7fa531a (14 August 2026).

pipeline path gaps - end-to-end flow

Plain-English explanation

The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls.

How it works technically

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The devops boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Repository gap. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Snapshot: main @ 3225f7fa531a (14 August 2026).

pipeline path gaps - errors and edge cases

Plain-English explanation

The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls.

How it works technically

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The devops boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Repository gap. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Snapshot: main @ 3225f7fa531a (14 August 2026).

pipeline path gaps - deployment, security and operational validation

Plain-English explanation

The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls.

How it works technically

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The devops boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Repository gap. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Snapshot: main @ 3225f7fa531a (14 August 2026).